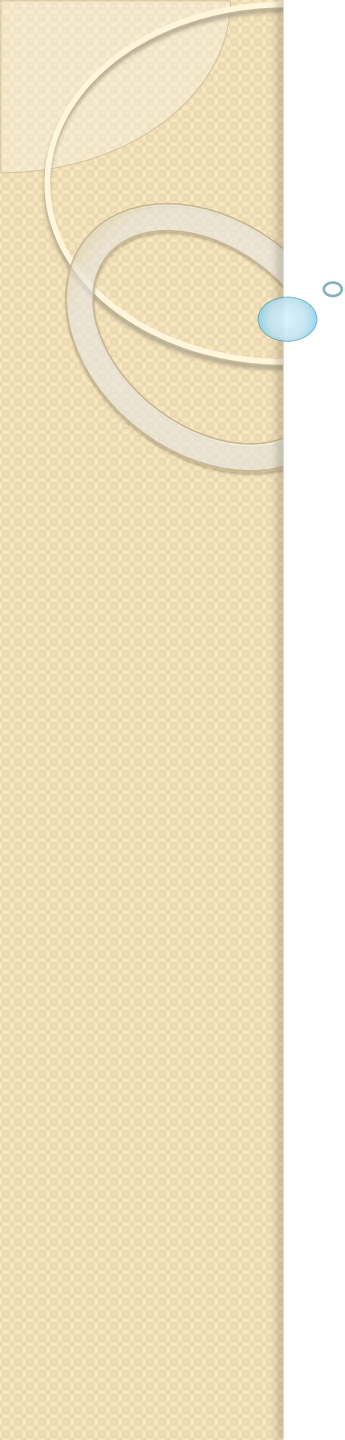




Chapter-6

Database Query Languages

Prepared by: Marta G. (MSc.)

Contents

- Relational Algebra
- Relational Calculus
- Basic SQL Queries
 - Data Definition Language
 - Data Manipulation Language
- Create user and grant permission
- Create triggers

Introduction

- Query is a question or requesting information.
- **Query language is a language which is used to retrieve information from a database.**
- Query language is divided into two types :
 - **Procedural language**
 - **Non-procedural language**

Procedural language

- Information is retrieved from the database **by specifying the sequence of operations to be performed.**

For Example: **Relational algebra.**

- Structure Query language (SQL) is based on relational algebra.

Non-procedural language

- Information is retrieved from the database **without specifying the sequence of operation** to be performed. Users only specify what information is to be retrieved.
 - Example: **Relational Calculus**
- Relational calculus is a **non-procedural query language** in which information is retrieved from the database **without specifying sequence of operation** to be performed.
- It tells what to do but **never explains how to do it**.
- Relational calculus is of two types which are as follows:
 - Tuple calculus
 - Domain calculus

Relational Algebra

- It is a procedural query language.
- Relational algebra consists of a set of operations that **take one or two relations** as an **input** and **produces a new relation as output**.
- It uses operators to perform queries which may be **unary(it takes one relation as an input)** or **binary (takes two relations)**.
- Select, project, rename comes under unary operation (operate on one table). Union, intersection, difference, cartesian, join, division comes under binary operation (operate on two tables).
- Relational algebra **targets how to obtain the result**.
- **Fundamental operations**
 - Select: $R1 := \sigma C(R2)$
 - Project: $R1 := \pi L(R2)$
 - Union: $R3 = R1 \cup R2$
 - Set difference: $R3 = R1 - R2$
 - Cartesian product: $R3 = R1 * R2$
 - Rename: $R1 := \rho R1(A1, \dots, An)(R2)$
- **Additional operations**
 - Set intersection ($\cap$)
 - Assignment operation ($\leftarrow$)
 - Natural join ($\bowtie$)
 - Division operation ($\div$)
 - Join operations
 - Inner join
 - Outer join

Cont...

The **SELECTION** operation

- It is **Unary** relational operations
- Chooses the **subset of tuples** from the relation that satisfies the given condition mentioned in the syntax of selection.
- The selection operation is also known as **horizontal partitioning** since it partitions the table or relation horizontally.
- Notation: $\sigma_{\text{condition}}(R)$
 - ‘ σ (sigma)’ is used to denote select(choose) operator
 - Condition like $\text{Id} = 3$ or combination of condition like $\text{age} > 20 \text{ AND } \text{CGPA} > 3.5$
 - R represents relation name from which you want to choose or select

Cont...

Employee			
Id	Name	Experience	Salary
05	Abebe	5 years	18,000
06	Kebede	3 years	16,000
07	Almaz	2 years	14,000
08	Hana	7 years	20,000

Eg1.: Select the employee tuples whose experience is greater than five or salary is greater than 18,000

$\sigma_{\text{experience} > 5 \text{ OR salary} > 18,000}$ (Employee)

Id	Name	Experience	Salary
05	Abebe	5 years	18,000
08	Hana	7 years	20,000

Cont...

The PROJECTION operation:

- It is also a unary operation which displays the **specific column** of a table mentioned in the query.

Syntax: $\Pi_{\langle \text{attribute_list} \rangle} (R)$

Eg2: list employees name and salary from the above employee table

Soln: $\Pi_{\text{Name, Salary}} (\text{Employee})$

Name	Salary
Abebe	18,000
Kebede	16,000
Almaz	14,000
Hana	20,000

Cont...

Eg3: select name and experience of employees whose salary is greater than 17,000

$\Pi_{\text{Name, Experience}} (\sigma_{\text{salary} > 17,000} (\text{Employee}))$

Id	Name	Experience	Salary
05	Abebe	5 years	18,000
08	Hana	7 years	20,000

Resulting table

Name	Experience
Abebe	5 years
Hana	7 years

Cont...

The RENAME operation (ρ)

- It is a unary relational operations that is **used to rename relation and/or column names**
- Reasons to rename a relation can be many, like
 - We may want to save the result of a relational algebra expression as a relation so that we can use it later.
 - We may want to join a relation with itself, in that case, it becomes too confusing to specify which one of the tables we are talking about, in that case, we rename one of the tables and perform join operations on them.

Cont...

Syntax:

$\rho_{N(A1,A2,...,An)}(R)$

OR

$\rho_N(R)$

OR

$\rho_{(A1,A2,...,An)}(R)$

Where R: Old relation name

N: New relation name

A1,A2,...,An: New attribute name

Cont...

Eg1: Rename relation Employee to New_employee

$\rho_{\text{New_employee}}(\mathbf{Employee})$

Eg2: write a query to rename relation Employee as Emp and attributes of Employee: Name and Salary as FirstName and Payment where salaries are less than 18,000 .

$\rho_{\text{Emp}}(\text{FirstName, Payment}) \pi_{\text{Name, Salary}}(\sigma_{\text{salary} < 18,000}(\text{Employee}))$

Cont...

The Cartesian Product(X/*)

- It is a binary set operation means, at a time we can apply the operation on two relations.
- It used **to merge columns from two relations**.
- Also called cross product or cross join

Eg1: Student X Details

Student

Id	Name
01	kebede
02	Abebe

Details

Roll_no	Dept	Age
11	SE	20
12	CS	22

Id	Name	Roll_no	Dept	Age
01	Kebede	11	SE	20
02	Abebe	11	SE	20
01	Kebede	12	CS	22
02	Abebe	12	CS	22

Cont...

- Generally, a cartesian product is never a **meaningful operation when it performs alone**. However, it becomes meaningful when it is followed by other operations.
- **Eg:** $\sigma_{\text{Roll_no}=11}(\text{Student X Details})$

Id	Name	Roll_no	Dept	Age
01	Kebede	11	SE	20
02	Abebe	11	SE	20

Intersection operation

- It displays the common values in both relations. It is denoted by $\cap$.
- Eg: Find a person who is student and lecturer

Extention_Stud

Id	Name
001	Abebe
002	Kebede
003	Haleta

Lecturer

Id	Name
003	Haleta
004	Beza
008	Samuel

Extention_Stud $\cap$ Lecturer

Id	Name
003	Haleta

Cont...

Set Difference (−)

- The result of set difference operation is tuples, which are present in one relation but are not in the second relation.
- **Eg:** $\Pi_{\text{author}}(\text{Books}) - \Pi_{\text{author}}(\text{Articles})$
- **Result:** Provides the name of authors who have written books but not articles.

The JOIN operation ($\bowtie$)

- It is a binary operation
- **Join** is a **combination** of a **Cartesian product** followed by a **selection process**.
- A Join operation **pairs two tuples** from different relations, if and only if a **given join condition is satisfied**.
- The tables in DBMS are associated using the primary key and foreign keys.
- **Syntax:** $R \bowtie_{\text{join_condition}} S$
- There are mainly two types of joins in DBMS:
 1. Inner Joins: Theta, Natural, EQUI
 2. Outer Join: Left, Right, Full

The JOIN operation (⋈)

cont...

Inner join

- It is used to **return rows from both tables which satisfy the given condition**. It is the most widely used join operation and can be **considered as a default join-type**
- An Inner join or equijoin is a comparator-based join which uses equality comparisons in the join-predicate.
- However, if you use other comparison operators like “>” it can’t be called equijoin.

The JOIN operation ($\bowtie$)

cont...

Theta Join

- It allows you to **merge two tables based on the condition** represented by theta.
- Theta joins work for **all comparison** operators.
- It is denoted by symbol θ . The general case of JOIN operation is called a Theta join.

Join_condition: $A_i \theta B_j$

Syntax: $R1 \bowtie_{A_i \theta B_j} R2$

Where A_i : Attributes of R1

B_j : Attributes of R2

$\theta : \{=, <, <=, >, >=, \neq\}$

Cont...

R1

Name	A2
Hana	35
Dawit	40
Solomon	45

R2

A3
25
35
20

Eg. Find $R1 \bowtie_{A2 \leq A3} R2$

First: Cartesian operation performed

Name	A2	A3
Hana	35	25
Hana	35	35
Hana	35	20
Dawit	40	25
Dawit	40	35
Dawit	40	20
Solomon	45	25
Solomon	45	35
Solomon	45	20

Secondly: THETA join performed

Name	A2	A3
Hana	35	35

The JOIN operation ($\bowtie$)

cont...

Equijoin

- When Theta join uses only **equality** comparison operator, it is said to be equijoin.

The JOIN operation (⋈)

cont...

The NATURAL JOIN operation (⋈/*)

- It does not use **any comparison operator**.
- It does not concatenate the way a Cartesian product does.
- We can perform a Natural Join **only if there is at least one common attribute** that exists between two relations.
- In addition, the **attributes must have the same name and domain**.
- Find **Courses** ⋈ **HoD** from the following tables

Courses

CourseCode	Course	Dept
SeEngi2021	DB	SE
CoSc2045	IP	CS

HoD

Dept	Dept_Head
SE	Temesgen
CS	Seid

Courses ⋈ HoD

CourseCode	Course	Dept	Dept_Head
SeEngi2021	DB	SE	Temesgen
CoSc2045	IP	CS	Seid

Cont...

Eg: What if you are asked to make natural join between employee and detail table

Employee

Emp_Id	Roll	TeamNo.
01	QA	1
02	Developer	2

Detail

No.	Salary
1	40,000
2	80,000

We need to rename the column first

Employee_Detail $\leftarrow$ Employee * $\rho_{(\text{TeamNo.}, \text{Salary})}$ (**Detail**)

Result

Emp_Id	Roll	Team_no	Salary
01	QA	1	40,000
02	Developer	2	80,000

Outer Joins

- An inner join includes only those tuples with **matching attributes** and the **rest are discarded** in the resulting relation.
- Therefore, we need to use outer joins to include all the tuples from the participating relations in the resulting relation.
- There are three kinds of outer joins: left outer join, right outer join, and full outer join.

Cont...

Left Outer Join($R \bowtie S$)

- All the tuples from the Left relation, RL, are included in the resulting relation.
- If there are tuples in RL without any matching tuple in the Right relation RR, then the RR-attributes of the resulting relation are made NULL.

RL

A	B
100	Database
101	Mechanics
102	Electronics

RR

A	B
100	Alex
102	Maya
104	Mira

$RL \bowtie RR$ will be

A	B	C	D
100	Database	100	Alex
101	Mechanics	-----	-----
102	Electronics	102	Maya

Cont...

Right Outer Join: ($R \bowtie_r S$)

- All the tuples from the Right relation, RR, are included in the resulting relation.
- If there are tuples in RR without any matching tuple in RL, then the RL-attributes of resulting relation are made NULL.
- $RL \bowtie_r RR$ will be:

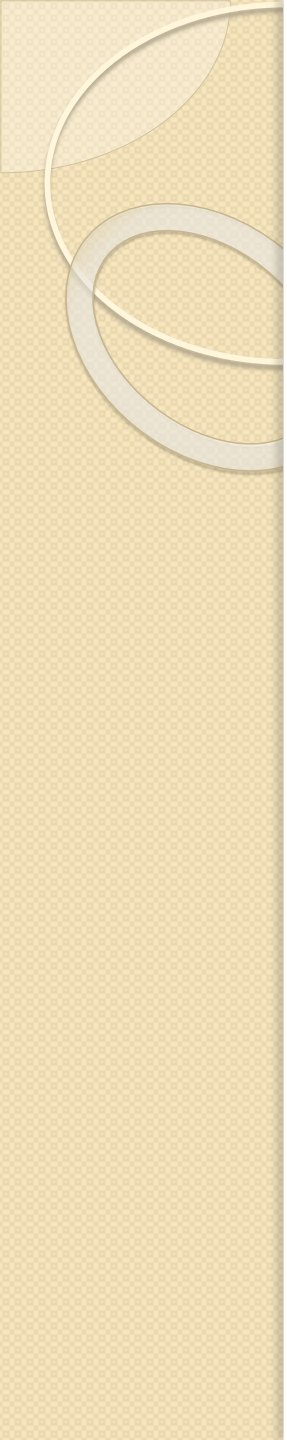
A	B	C	D
100	Database	100	Alex
102	Electronics	102	Maya
-----	-----	104	Mira

Cont...

Full Outer Join: ($R \bowtie S$)

- All the **tuples** from **both participating relations** are **included** in the **resulting relation**.
- If there are no matching tuples for both relations, their respective **unmatched attributes** are made **NULL**.
- $RL \bowtie RR$

A	B	C	D
100	Database	100	Alex
101	Mechanics	-----	-----
102	Electronics	102	Maya
-----	-----	104	Mira



What is DEVISION operation?

Non-Procedural language

- Information is retrieved from the database **without specifying the sequence of operation** to be performed. Users only specify what information is to be retrieved.
- Example: Relational Calculus
- Relational calculus is a **non-procedural query language** in which information is retrieved from the database without specifying sequence of operation to be performed.
- It tells what to do but **never explains how to do it**.
- Relational calculus is of two types which are as follows –
 - Tuple calculus
 - Domain calculus

Non-Procedural language

Tuple calculus

- It describes the desired information without giving a specific procedure for obtaining that information.
- The tuple relational calculus is specified to select the tuples in a relation. In TRC, **filtering variable uses the tuples of a relation**. The result of the relation can have one or more tuples.
- Variable range over tuple like sql
- Notation: $\{T \mid P(T)\}$ or $\{T \mid \text{Condition}(T)\}$

Where

- **T** is the resulting tuples
- **P(T)** is the condition used to fetch T.

Cont...

- **Example:** To find the first and last names of all employees whose salary is above \$50,000, we can write the following tuple calculus expression:

$\{t.FNAME, t.LNAME \mid EMPLOYEE(t) \text{ AND } t.SALARY > 50000\}$

- The condition $EMPLOYEE(t)$ specifies that the **range relation** of tuple variable t is $EMPLOYEE$.
- The first and last name of each $EMPLOYEE$ tuple t that satisfies the condition $t.SALARY > 50000$ ($SELECTION \sqsubseteq_{SALARY > 50000}$) will be retrieved.

Cont...

- Example: { T.name | Author(T) AND T.article = 'database' }

Output: This query selects the **tuples from the AUTHOR relation**. It returns a tuple with '**name**' from **Author** who has **written** an **article** on '**database**'.

- TRC (tuple relation calculus) are **free variables** unless can be quantified. So that, we can use **Existential ($\exists$)** and **Universal Quantifiers ($\forall$)**.
- The phrase "**for every x**" (sometimes "**for all x**") is called a universal quantifier and is denoted by $\forall x$.
- The phrase "**there exists an x such that**" is called an existential quantifier and is denoted by $\exists x$.

To remember

- $\forall x (x \text{ is a square} \Rightarrow x \text{ is a rectangle})$, i.e., "all squares are rectangles."
- $\forall x (x \text{ lives in Walla Walla} \Rightarrow x \text{ lives in Washington})$, i.e., "every person who lives in Walla Walla lives in Washington."
- $\exists x (x \text{ is a professor} \wedge x \text{ is a republican})$, i.e., "some professor is a republican."
- $\exists x (x \text{ is a prime number} \wedge x \text{ is even})$, i.e., "some prime number is even."

Cont...

Eg:

$\{ R \mid \exists T \in \text{Authors}(T.\text{article} = \text{'database'} \text{ AND } R.\text{name} = T.\text{name}) \}$

Output: This query will yield the same result as the previous one.

Cont...

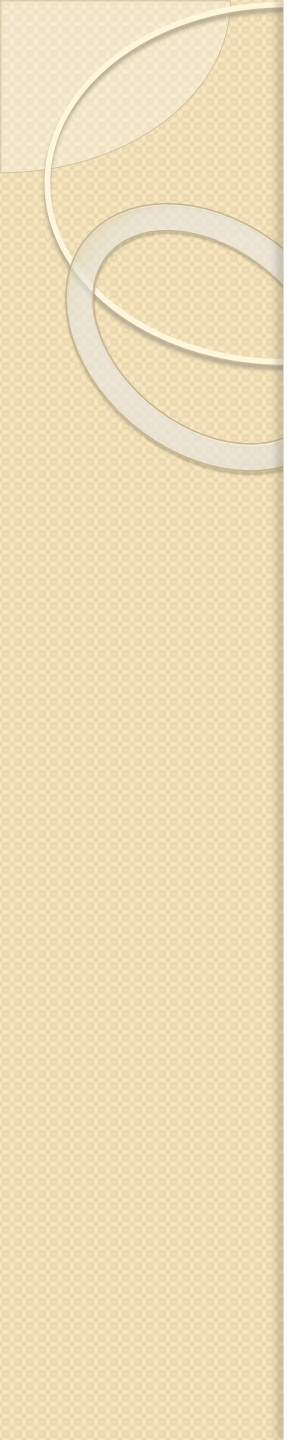
Domain calculus

- Variable range over domain(particular element)
- **Filtering variable uses the domain of attributes.**
- Domain relational calculus uses the same operators as tuple calculus. It uses logical connectives $\wedge$ (and), $\vee$ (or) and $\neg$ (not).
- It uses Existential ($\exists$) and Universal Quantifiers ($\forall$) to bind the variable.

Notation: $\{ a_1, a_2, a_3, \dots, a_n \mid P(a_1, a_2, a_3, \dots, a_n) \}$

Where: a_1, a_2 are attributes

P stands for formula built by inner attributes



Eg: $\{ \langle \text{article, page, subject} \rangle \mid \in \text{javatpoint} \wedge \text{subject} = \text{'database'} \}$

Output: This query will yield the article, page, and subject from the relational javatpoint, where the subject is a database.

SQL (Structured Query Language)

- It is a **computer language** for **storing, manipulating** and **retrieving** data stored in relational database.
- SQL is an ANSI (American National Standards Institute) standard, but there are many different versions of the SQL language.
- SQL is the **standard language** for **Relation Database System**.
- All relational database management systems like MySQL, MS Access, Oracle, Sybase, Informix, postgres and SQL Server **use SQL as standard database** language.
- **Query:** question we ask to our database/tables.
- SQL **uses** the terms **table, row, and column** for **relation, tuple, and attribute**, respectively.

Why SQL?

- Allows users to access data in relational database management systems.
- Allows users to describe the data.
- Allows users to define the data in database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create view, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views
- SQL is an ANSI standard computer language
- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert new records in a database
- SQL can delete records from a database
- SQL can update records in a database
- SQL is easy to learn

SQL (Structured Query Language)

- Considered as one of the major reasons for the success of relational databases in the commercial world.
- It is helpful to use **DDL** and a **DML**.
- In addition, it facilities for defining views on the databases.

Query: question we ask to our database/tables.

- Block basically contains a SELECT-FROM-WHERE expression.
- SQL **uses** the terms **table**, **row**, and **column** for **relation**, **tuple**, and **attribute**, respectively.

DDL (Data Definition Language)

Basic DDL commands we usually use are: CREATE, DROP, ALTER, TRUNCATE and RENAME

CREATE:

- Creating schema/database → create database/schema < database name >;
- Creating table → create table < tablename(atr1 datatype, atr2 datatype,) >; or create table < dbname>.<tablename(atr1 datatype, atr2 datatype,...)>;

DROP:

- drop schema/database < schema name>;
- drop table < table name>;

Truncate:

- truncate table < table name>;

CONT...

ALTER:

- The definition of a base **table can be changed** by using the ALTER TABLE command, which is a schema evolution command.
- The possible alter actions include **adding or dropping a column(attribute), changing a column** definition, and **adding or dropping table constraints**.

- alter table < table name>

rename column < columnone > to <columntwo >;

- alter table < table name>

add column < attribute name >datatype;

- alter table < table name>

drop < attribute name >;

- To add constraint → alter table 'tablename' MODIFY 'attributename datatype' 'constraint name';

DML(Data Manipulation Language)

- SELECT, PROJECT, JOIN, INSERT, DELETE, UPDATE...
- SQL has one basic statement for **retrieving information** from a database: the **SELECT** statement.
- SQL table may not be a set of tuples (when the table is created without primary key constraint), because a set does not allow two identical members.
- SQL relations are constrained to be sets when a key constraint is used in the table creation or when the DISTINCT option is used with the SELECT statement.

Cont...

SELECT-FROM-WHERE Structure of SQL Queries:

- The basic form of **SELECT statement**, sometimes called a **mapping or a select-from-where block**, is **formed** of the **three clauses SELECT, FROM, and WHERE** and has the form:
- `SELECT < attribute list> FROM < table list> WHERE <condition>;`
- e.g; Retrieve the birthdate and address of the employee(s) whose name is "John B. Smith"
- Answer → `SELECT BDATE, ADDRESS FROM EMPLOYEE WHERE FNAME='John' AND MINIT='B' AND LNAME='Smith';`
- The above statement is similar to the following relational algebra expression π BDATE, ADDRESS (σ FNAME='John' AND MINIT='B' AND LNAME='Smith' (EMPLOYEE))

Cont...

- Hence, a simple SQL query with a single relation name in the FROM-clause is similar to a SELECT-PROJECT pair of relational algebra operations.
- The SELECT-clause of SQL specifies the projection attributes, and the WHERE-clause specifies the selection condition.
- Enforcing referential integrity constraints and implementing ON UPDATE, AND ON DELETE CASCADE.
- e.g; create table teacher(tid int primary key, tname varchar(25)); let's use tid as foreign key for student table
- **create table** student(sid int primary key, sname varchar(45) NOT NULL, SCGPA double, tid int, foreign key(tid) references teacher(tid) ON UPDATE CASCADE);
- **create table** student(sid int primary key, sname varchar(45) NOT NULL, SCGPA double, tid int, foreign key(tid) references teacher(tid) ON DELETE CASCADE);

Cont...

- Dealing with ambiguous attribute names
- In SQL the **same name can be used for two or more attributes** as long as the **attributes are in different relations**.
- If this is the case, and a query refers to two or more attributes with the same name, we **must qualify the attribute name with the relation name**, to prevent ambiguity.
- This is done by **prefixing the relation name** to the **attribute name** and **separating the two by a period**.
- e.g; Suppose that the Department Number and Last Name attributes of the EMPLOYEE relation were called 'DNUMBER' and 'NAME', and the Department Name attribute of DEPARTMENT was also called 'NAME';
- Then, to prevent ambiguity, the above statement would be rephrased as:

Cont...

We must **prefix the attributes** NAME and DNUMBER in the previous example to specify which ones we are referring to, because the attribute NAME is used in both relations:

Query: SELECT FNAME, **EMPLOYEE.NAME**, ADDRESS FROM
EMPLOYEE, DEPARTMENT WHERE
DEPARTMENT.NAME='RESEARCH' AND
DEPARTMENT.DNUMBER=EMPLOYEE.DNUMBER;

Reducing long query by using Alias

- Ambiguity also arises in the case of queries that refer to the same relation twice, as in the following.
- E.g; For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor
- Q: SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME FROM EMPLOYEE AS E, SUPERVISOR AS S WHERE E.SUPERSSN=S.SSN;
- In this case, we are allowed to **declare alternative relation names** E and S, called **aliases or tuple variables**, for the EMPLOYEE and SUPERVISOR relations.
- An alias can follow the **keyword AS**, as shown above or it can directly follow the relation name for example, by writing EMPLOYEE E, SUPERVISOR S in the WHERE-clause as the above query.

Some Additional Query Examples

Retrieve distinct attributes from a given table:

- To **select distinct names** from student table
select distinct name from student;
- To **retrieve the sum of salary** of employees
select sum(salary) from employee;
- To display only a single set cgpa of student table
select all cgpa from student;
- To join student, teacher and manager tables without any condition
select * from student, manager, teacher or select * from student join manager join teacher;
- Try left join, right join, full join.
- **What is inner join? What is outer join? What is natural join?**

Substring Pattern Matching and Arithmetic Operators

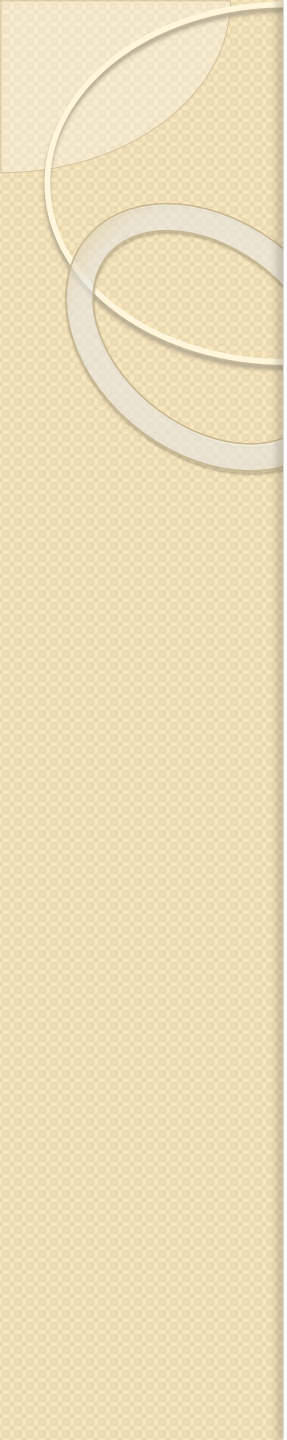
- Use comparison conditions on only parts of a character string, using the **LIKE** comparison operator → used for string pattern matching.
- Partial strings are specified using two reserved characters: modulo (%) replaces an arbitrary number of zero or more characters, and the underscore (_) replaces a single character.
- For example, consider the following query.
- E.g: `select * from student where department LIKE '%soft%';`
- The above query displays all students whose department likes soft.
- To display students with cgpa having 7 at third place in their grade → `select * from student where cgpa like ' _ 7 _';`

Cont...

- ✓ To display students with cgpa between 3.6 and 4.0;
- ✓ Using **BETWEEN** to show ranges → select * from student where (cgpa between 3.6 and 4.0);
- ✓ Select maximum grade from student table → select max(cgpa) from student;
- ✓ Select sum of student CGPA → select sum(cgpa) from student;

Nested Queries

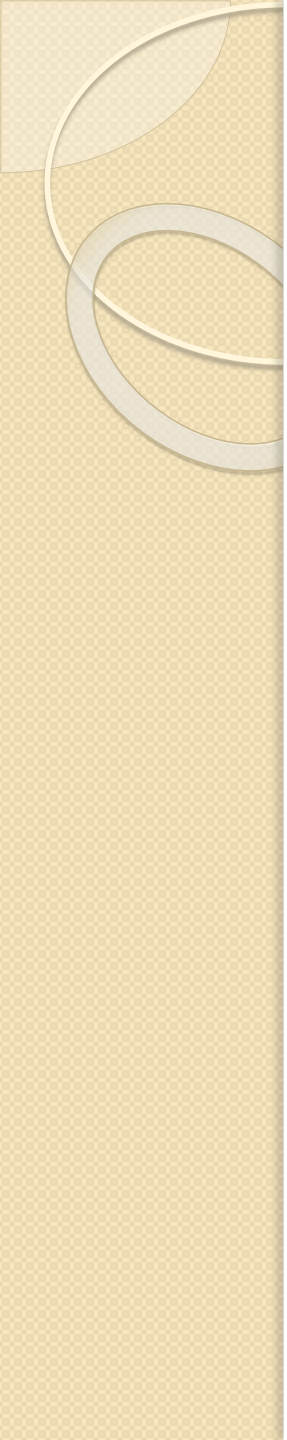
- It is defining **queries inside queries**.
- E.g; select name, id from employee where salary=(select MAX(salary) from employee);



Reading and practical assignment

How to Create user and grant permission using sql?

How to Create Trigger?



Thank you !!!

Class quiz

1. What is the difference between procedural and non- procedural languages?
2. List and describe unary relational algebras with there symbol
3. List types of relational calculus
4. Describe join relational algebra operation

Employee			
Id	Name	Experience	Salary
05	Abebe	5 years	18,000
06	Kebede	3 years	16,000
07	<u>Almaz</u>	2 years	14,000
08	Hana	7 years	20,000

Based on the above table

5. Select salary and name who has greater or equal to 5 years

Of experience using relational algebra

6. Rename ID and experience as idno and working_year respectively